C++ Programming

Juha Yli-Hemminki

06.11.2025

SeAMK

Structs

- Way to define own types by combining multiple variables:

```
struct myStructure
{
        int value1;
        int value2;
}

myStructure ms = {10,20}; // initial values given here
ms.value1 = 15; // values can be used and changed one by one
```

Structs

- All members are public by default
- In C++ functions can be added into Struct as well:

```
struct myStructure
{
        int value1;
        int value2;

        void printValues()
        {
                std::cout << value1 << ” ” << value2 << std::endl;
        }
}
```

Classes

- C++ (C with classes) is object-oriented language
- Classes may refer to real life objects
  - For example ‘bike’
  - In real life bike has attributes like brand, model etc. and functions like brake
  - In Class these attributes are variables and functions are methods that belong to the class (class members)

Classes

- Class is user defined data type. It can be used like basic types (int, double):

```
class myClass
{
public:                // Access specifier
        int value1;
        int value2;
}
```

- Class members are **private** by default
  - Not accessible from outside of the class

Classes

- To create static object:

```
myClass myObject;
myObject.value1 = 15;
```

- To create dynamic object (allocate memory at run time):

```
myClass* myObject = new myClass(); // new returns a pointer to created object
myObject->value1 = 15;
```

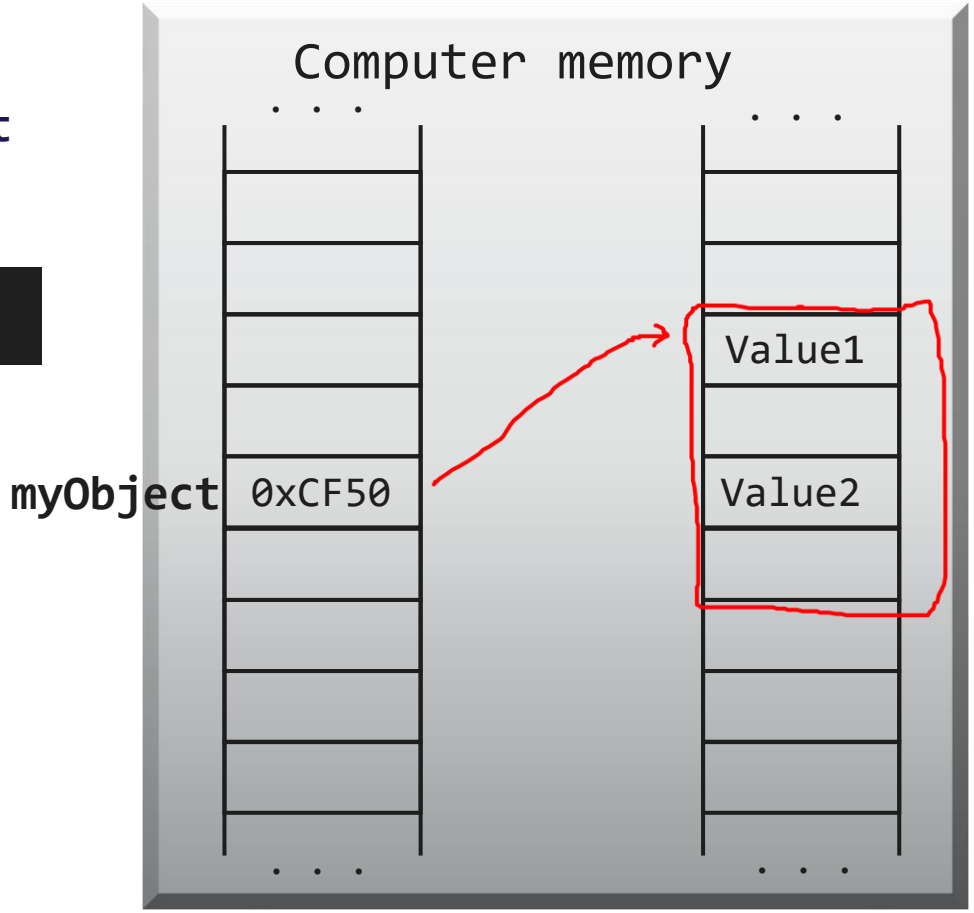
SeAMK

Object in memory

Address to new object is stored in variable **myObject**

```
myClass* myObject;
myObject = new myClass();
```

Address to myClass type object in computer memory



Object made of class **myClass** in address 0xCF50

Constructor

- Constructor of the class is called when new object is created:
  - Like method but it has same name as a class

```
class myClass
{
public:                 // access specifier
        int value1;
        int value2;

        myClass() // constructor
        {
        }
}
```

Constructor

- It’s possible to have multiple constructors with different input values (overloading):

```
class myClass
{
public:                 // access specifier
        int value1;
        int value2;

        myClass() // constructor
        {
        }
        myClass(int a, int b) {} // another constructor
}
```

Destructor

- Destructor is called when the object is deleted:

```
class myClass
{
public:                 // access specifier
        int value1;
        int value2;

        myClass() // constructor
        {
        }
        ~myClass() {} // destructor
}
```

Deleting object

- Dynamically allocated objects must be deleted

```
myClass* myObject = new myClass();
// new returns a pointer to created object
delete(myObject);
// deletes object. Destructor is called
```

SeAMK

SEINÄJOEN AMMATTIKORKEAKOULU

SEINÄJOKI UNIVERSITY OF APPLIED SCIENCES